Detectio 模块踩踏的蓝队检测视角

- **本课目标**

1. 理解 EDR 检测 Module Stomping 的三种核心手段：

- 监控 WriteProcessMemory 写入 .text 节的行为**
- 检查 CreateRemoteThread 起始地址是否落在合法模块范围内**
- 对比内存与磁盘上的模块字节是否一致

2. 掌握至少两种反制策略：

- 用 APC（异步过程调用）替代 CreateRemoteThread
- 用回调函数执行 Shellcode

3. 对比 Module Stomping 与 VirtualAllocEx 方案的优缺点。

- **检测手段一：监控对 .text 节的写入**

(一) 为什么这是最强的检测点？

正常程序几乎从不修改自己的代码段。.text 节在加载时就应该是固定的、只读可执行的。如果一个进程试图往 .text 节里写数据，这本身就是异常行为。

(二) EDR 如何监控？

EDR 可以通过以下三种方式监控 .text 节的写入：

监控方式	物理动作
内核回调	注册 PsSetCreateProcessNotifyRoutine 或 ObRegisterCallbacks，在 WriteProcessMemory 到达内核时拦截
ETW 事件	启用 Microsoft-Windows-Kernel-Audit-API-Calls 提供程序，记录每一次 WriteProcessMemory 调用
内存页属性监控	定期扫描进程内存页，发现原本是 PAGE_EXECUTE_READ 的页被改成了 PAGE_READWRITE 或 PAGE_EXECUTE_READWRITE

(三) 检测逻辑

EDR 会检查：

- 目标地址是否落在某个已加载模块的 .text 节范围内？
- 写入的字节是否和磁盘上的原始文件不一致？

如果两个条件都满足 → 高危告警。

上一课的操作：

```
WriteProcessMemory(hProcess, pRemoteFunc, shellcode, ...);
// pRemoteFunc 落在 amsi.dll 的 .text 节
// 写入的 shellcode 和 amsi.dll 磁盘文件不一致
```

EDR 会看到："有人在修改 amsi.dll 的代码段，而且写进去的字节不是原始文件的内容。"

- **检测手段二：检查线程起始地址**

(一) EDR 如何检查？

EDR 监控 CreateRemoteThread（及更底层的 NtCreateThreadEx），查新线程的起始地址：

- 起始地址是否落在某个已加载模块的可执行段内？
- 这个地址是否是某个合法导出函数的入口？
- 这个地址是否和磁盘上的原始文件对应？

(二) Module Stomping 会被怎么判定

```
CreateRemoteThread(hProcess, NULL, 0, (LPTHREAD_START_ROUTINE)pRemoteFunc, ...);
// pRemoteFunc = amsi.dll!AmsiScanBuffer 的地址
```

EDR 的分析逻辑：

- 起始地址落在 amsi.dll 的可执行段内 ✅（正常）
- 起始地址是 AmsiScanBuffer 的入口 ✅（正常）
- 但是：AmsiScanBuffer 的原始代码是反病毒扫描逻辑，它不应该被作为线程入口调用。

EDR 的判定：“AmsiScanBuffer 被当作线程入口调用，这不是它的正常用途，可疑。”

(三) 对比：VirtualAllocEx 方案的检测

方案	起始地址位置	EDR 判定
VirtualAllocEx + CreateRemoteThread	新申请的内存（不在任何模块内）	高危：未知内存作为线程入口
Module Stomping + CreateRemoteThread	合法模块的可执行段	中危：合法模块内的函数被异常调用

结论：Module Stomping 的检测等级比 VirtualAllocEx 低，但不是零检测。

- **检测手段三：内存与磁盘的完整性对比**

(一) 物理动作

EDR 可以定期扫描进程内存，把已加载模块的.text 节内容和磁盘上的原始文件做对比。

- 如果内存里的字节和磁盘上的字节不一致 → 代码被修改了。
- 修改的位置如果落在 .text 节 → 高危。

(二) 这个检测的局限性

- 性能开销大：每次扫描都需要读取磁盘文件并逐字节对比。
- 不是实时：通常是定期扫描（比如每 30 秒），而不是实时拦截。
- 容易被绕过：如果 Shellcode 只修改了 .text 节末尾的 padding（Code Cave），而 padding 区在磁盘上也是 0x00，对比可能不会发现异常。

(三) 对 Module Stomping 的影响

覆写的是 AmsiScanBuffer 的真实代码，磁盘上有对应的字节。所以如果 EDR 做完整性对比，一定能发现异常。

反制思路：

- 选择不常被完整性扫描的 DLL（比如第三方 DLL，而不是系统核心 DLL）。
- 覆写后，在 Shellcode 末尾恢复原始字节（在 Shellcode 执行完后，把原函数代码写回去）。
- 使用 Hook 技术而不是完全覆写：只修改函数开头的一小段（比如 5 字节跳转指令），执行完再恢复。

- **反制策略一：用 APC 替代 CreateRemoteThread**

(一) 为什么 CreateRemoteThread 是最大暴露点

因为 EDR 重点监控“远程线程创建”这个行为。CreateRemoteThread 的调用链非常明显：

```
你的进程 → OpenProcess → CreateRemoteThread → 目标进程出现新线程
```

(二) APC 是什么

APC = Asynchronous Procedure Call（异步过程调用）。

它是 Windows 的一种机制：可以往一个线程的 APC 队列里插入一个函数，当这个线程进入“可警告等待状态”时，它会自动执行这个函数。

物理意义：不需要创建新线程，只需要“挂”一个函数到已有线程上。

(三) APC 注入的流程

- 找到目标进程的一个线程 ID（通过 CreateToolhelp32Snapshot）
- 用 OpenThread 打开这个线程

- 用 QueueUserAPC 把 Shellcode 地址插入到该线程的 APC 队列
- 等待该线程进入“可警告等待状态”（比如调用 SleepEx、WaitForSingleObjectEx）
- Shellcode 被自动执行

优势：

- 不创建新线程 → EDR 的“远程线程创建”监控点看不到。
- 执行发生在已有线程里 → 调用栈看起来更自然。

劣势：

- 需要目标线程进入可警告等待状态，不是所有线程都会进入。
- 需要在目标进程内存里写入 Shellcode（这一步和 Module Stomping 一样）。

(四) APC + Module Stomping 的组合

```
// ① 用 Module Stomping 把 Shellcode 写进 amsi.dll 的 .text
WriteProcessMemory(hProcess, pRemoteFunc, shellcode, shellcodeSize, NULL);
// ② 不用 CreateRemoteThread，改用 QueueUserAPC
HANDLE hThread = OpenThread(THREAD_SET_CONTEXT, FALSE, targetThreadId);
QueueUserAPC((PAPCFUNC)pRemoteFunc, hThread, 0);
CloseHandle(hThread);
```

效果：整个过程没有创建新线程，EDR 的“远程线程监控”完全失效。

- **反制策略二：用回调函数执行**

(一) 回调函数是什么

Windows 提供了很多“注册回调”的机制。你注册一个函数，系统在某个事件发生时自动调用它。常见的回调点：

- SetWindowsHookEx（键盘/鼠标钩子）
- EnumFontFamilies（枚举字体）
- CertEnumSystemStore（枚举证书）
- EnumChildWindows（枚举子窗口）

(二) 为什么回调能绕过检测

因为回调函数的执行不是由创建的线程触发的，而是由系统在正常流程中调用的。EDR 看到的是“系统在正常调用回调函数”，而不是“远程创建了线程”。

(三) 回调执行的典型流程

```
// ① 把 Shellcode 写进目标进程内存（Module Stomping 或 VirtualAllocEx）
WriteProcessMemory(hProcess, pRemoteFunc, shellcode, shellcodeSize, NULL);
// ② 注册回调，让系统在事件发生时调用它
EnumFontFamiliesA(hdc, NULL, (FONTENUMPROCA)pRemoteFunc, 0);
```

优势：

- 不创建远程线程。
- 执行点看起来是系统正常行为。
- 调用栈里没有你的进程。

劣势：

- 不同回调点的参数不一样，需要构造合适的调用上下文。
- 某些回调点已经被 EDR 重点监控（比如 SetWindowsHookEx）。

- **Module Stomping vs VirtualAllocEx：优劣对比**

对比项	VirtualAllocEx 方案	Module Stomping 方案
内存分配	申请新内存	复用已有内存

内存属性	需要 VirtualProtectEx 改成可执行	.text 节天然可执行
IAT 泄露	需要 VirtualAllocEx 等 API	无敏感 API
CFG 检查	不涉及（新内存不是 CFG 目标）	导出函数是 CFG 合法目标（放行）
EDR 检测	高：新内存 + 可执行属性 + 远程线程	中：.text 节被修改 + 远程线程
稳定性	高（不影响原程序）	低（覆写函数后原功能被破坏）
适用场景	通用注入	短时执行、不关心原函数是否被破坏

- **蓝队检测的最终逻辑（总结）**

检测点	蓝队看什么	红队反制
.text 节写入	是否有 WriteProcessMemory 写向可执行段	用 APC / 回调代替线程创建
远程线程起始地址	起始地址是否落在合法模块内	用 QueueUserAPC 替代 CreateRemoteThread
内存与磁盘对比	内存字节是否和磁盘一致	选择非核心 DLL，或在 Shellcode 后恢复原字节
调用栈分析	线程创建的调用栈是否有异常	用回调函数执行，调用栈更自然

- **本课小结**

➢ Module Stomping 的检测等级比 VirtualAllocEx 低，但不是零。
➢ 最大暴露点是 CreateRemoteThread：EDR 监控远程线程创建。
➢ 反制核心：用 APC 或回调替代远程线程创建。
➢ .text 节写入是另一个暴露点：EDR 监控对可执行段的修改。
➢ 红蓝对抗的本质：不是“谁技术更强”，而是“谁的特征更少”。